Created data source file : dogs.txt

Data sources are used to read the content of the infrastructure

for example if we want terraform to read the content of the file which has been created by any other tool.

create a file called in dogs.txt in the same terrafrom working directory.

main.tf

======

resource "local_file" "my-pet" {

filename = "pets.txt"

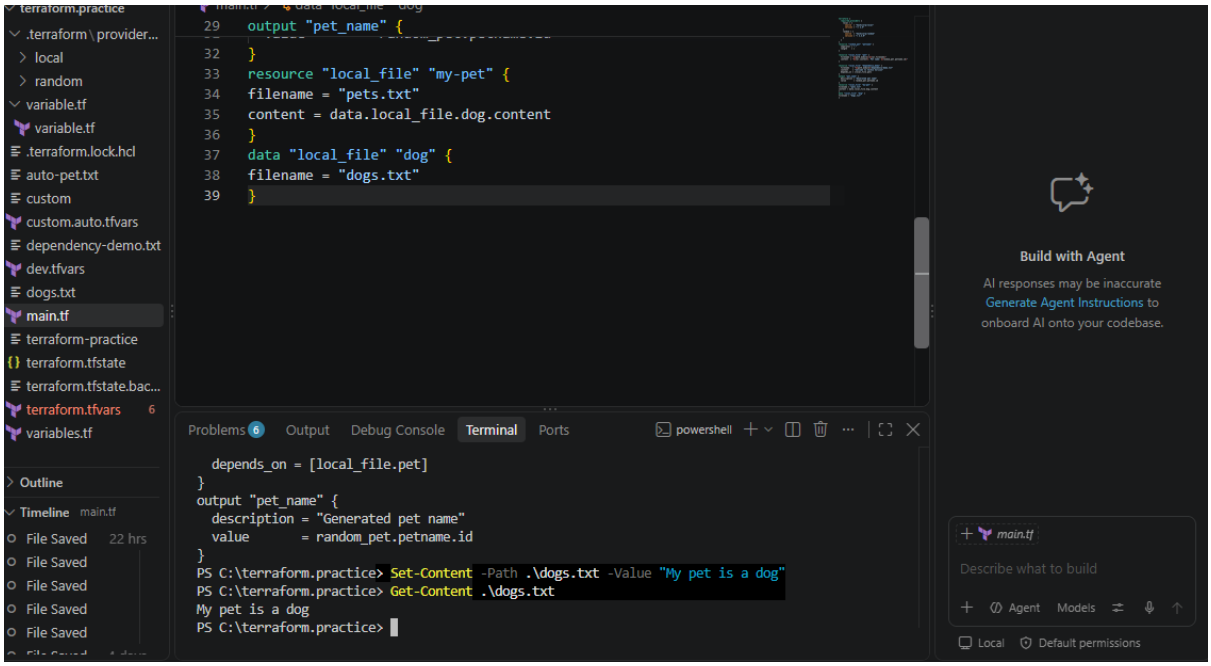
content = data.local_file.dog.content

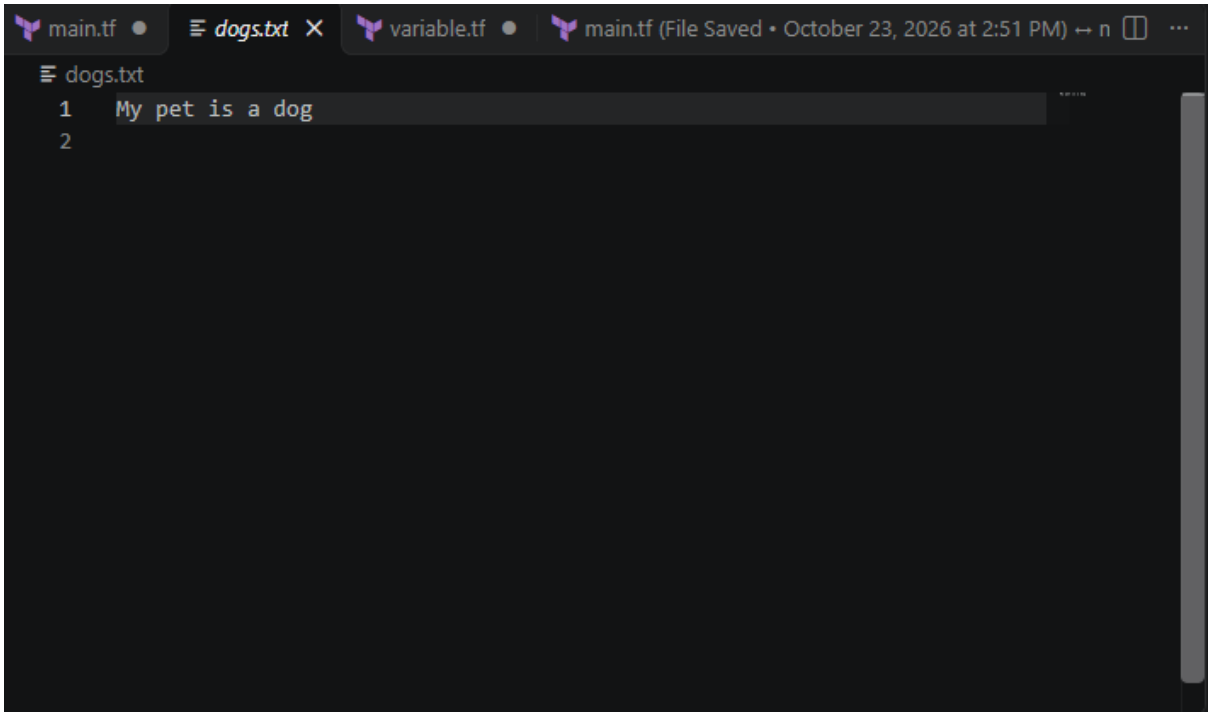
}

data "local_file" "dog" {

filename = "dogs.txt"

}





Meta-Arguments:

==========

Meta arguments are used if we want to create multiple resources.

Meta arguments can be used within any resource block to change the behaviour of the resources.

resource "local_file" "my-pet" {

filename = var.filename[count.index]

content = "I love cats!"

count = 3

}

variables.tf

variable "filename" {

default = [

"pets.txt"

"cats.txt"

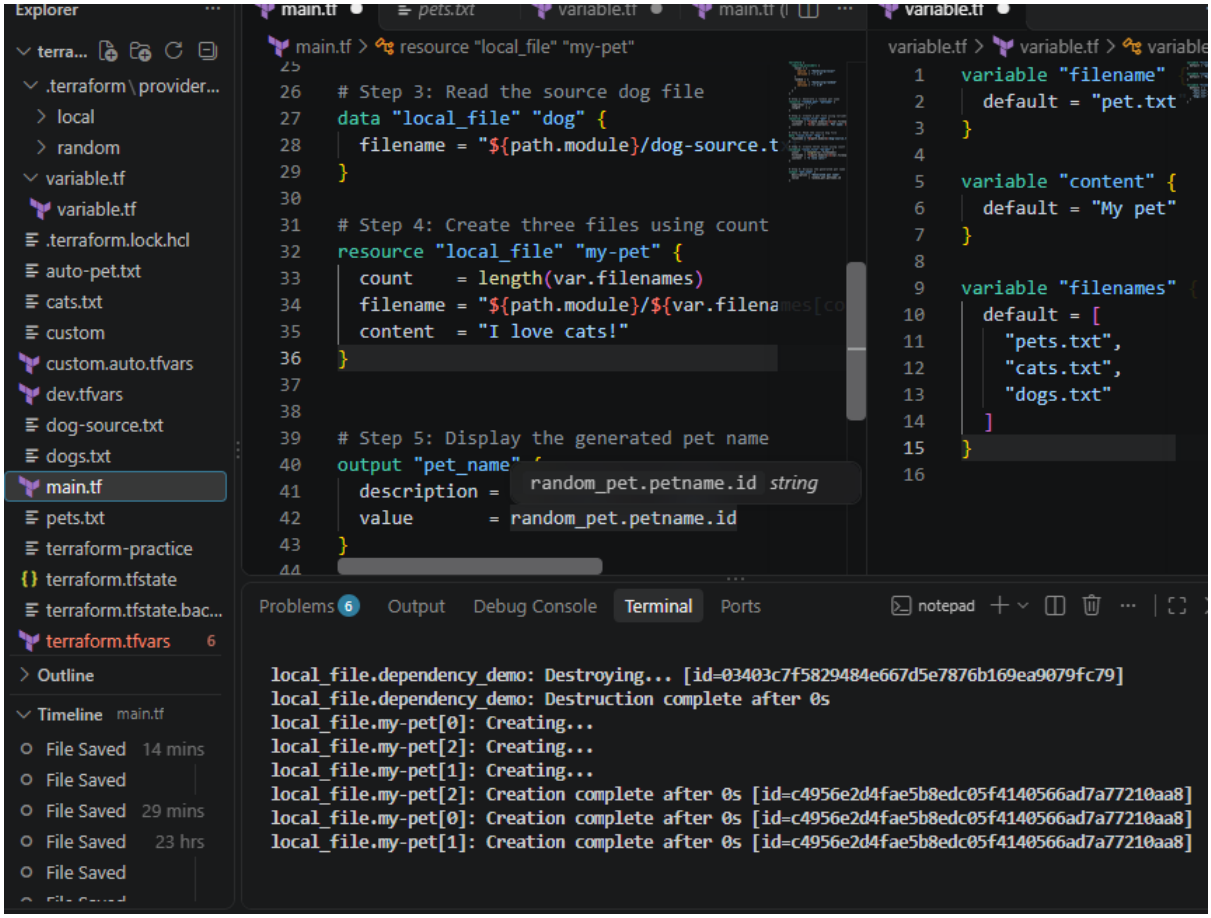
"dogs.txt"

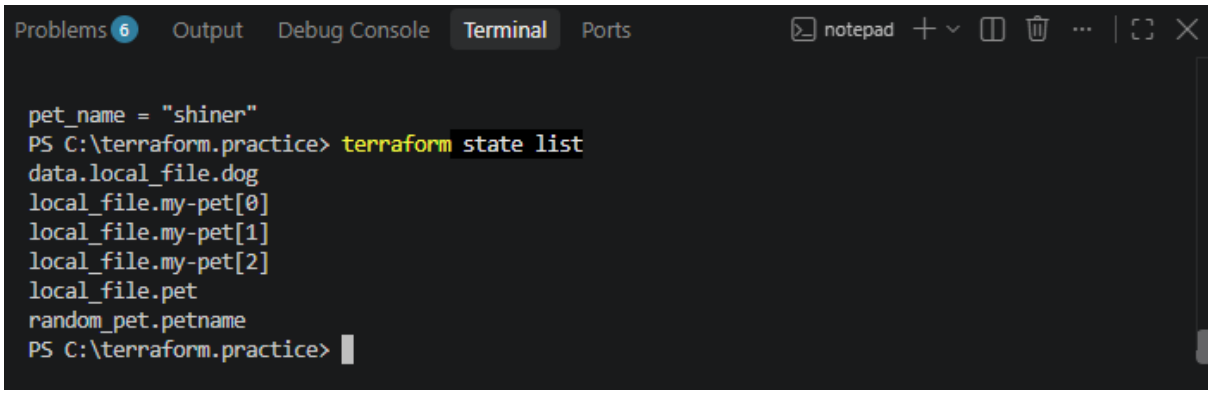
]

}

RESULT :

Successfully created 3 files while (using count 3)





- Defined a list variable containing three filenames.
- Used `length()` to determine the number of resource instances.
- Used `count.index` to select the appropriate filename.
- Executed Terraform validation and applied the configuration.

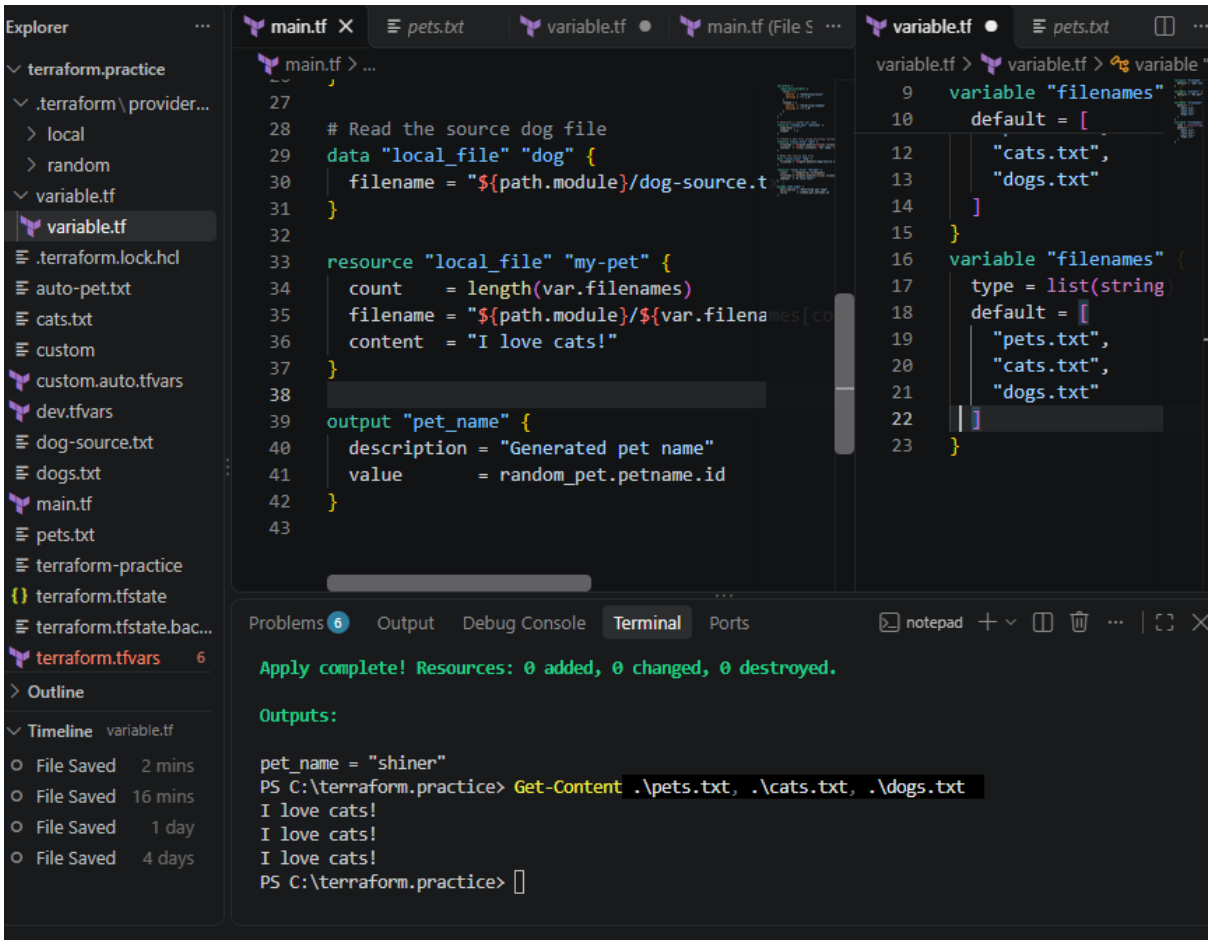
resource "local_file" "my-pet" {

filename = var.filename[count.index]

content = "I love cats!"

count = lenght(var.filename)

}



variables.tf

variable "filename" {

default = [

"pets.txt"

"cats.txt"

"dogs.txt"

]

}

- Created IAM user `lucy` using Terraform.
- Attached the AWS-managed `AdministratorAccess` policy.
- Verified the policy attachment through AWS CLI and the IAM console.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "ap-southeast-2"
}

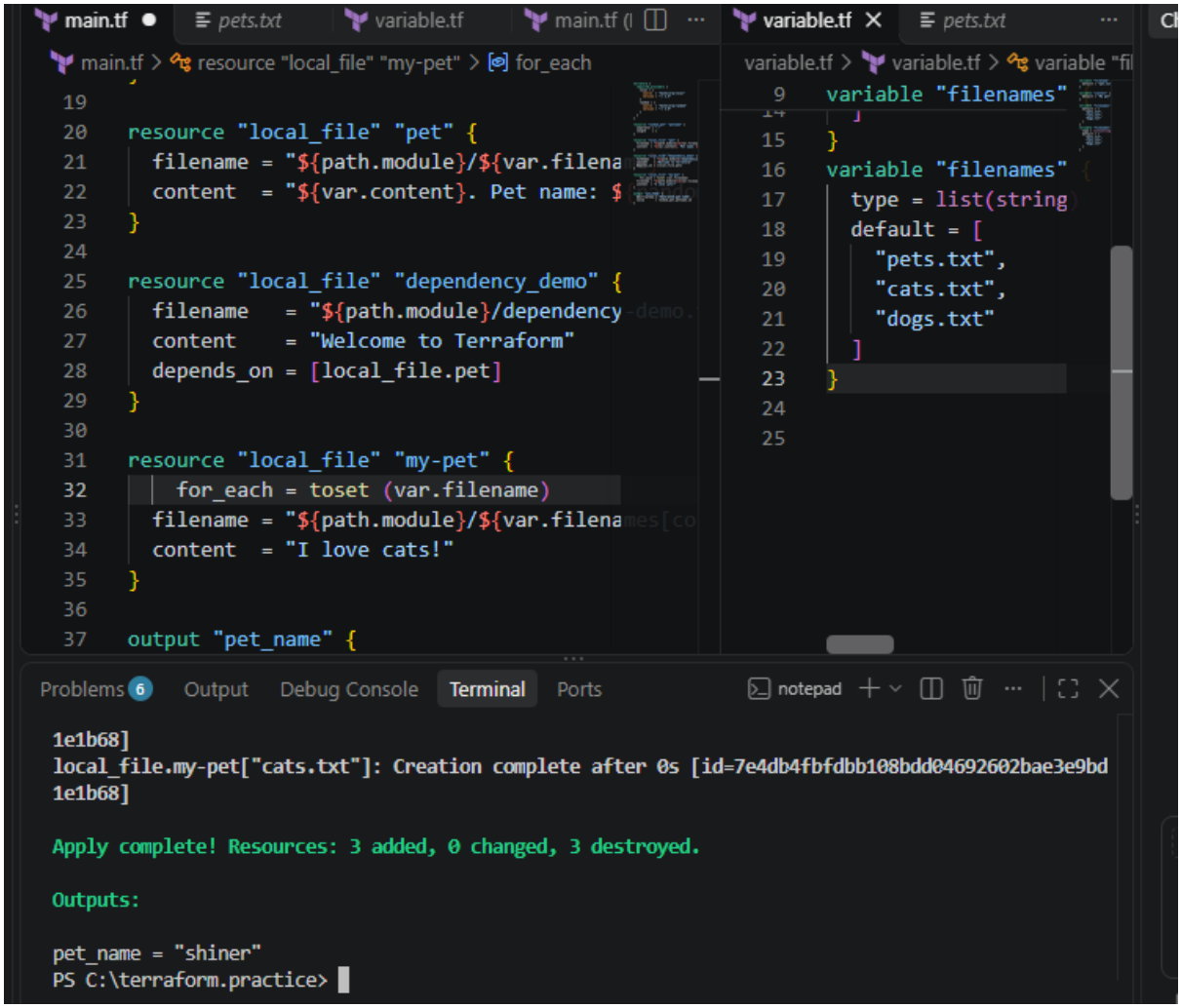
data "aws_caller_identity" "current" {}

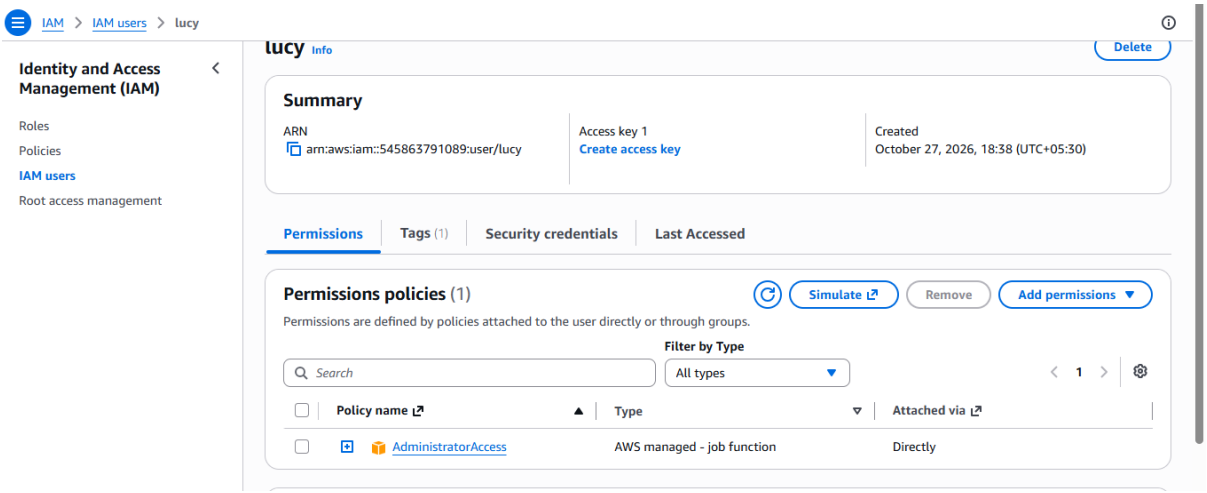
resource "aws_iam_user" "Admin-user" {
  name = "lucy"

  tags = {
    description = "Technical Team Lead"
  }
}

output "aws_account_id" {
  value = data.aws_caller_identity.current.account_id
}
```

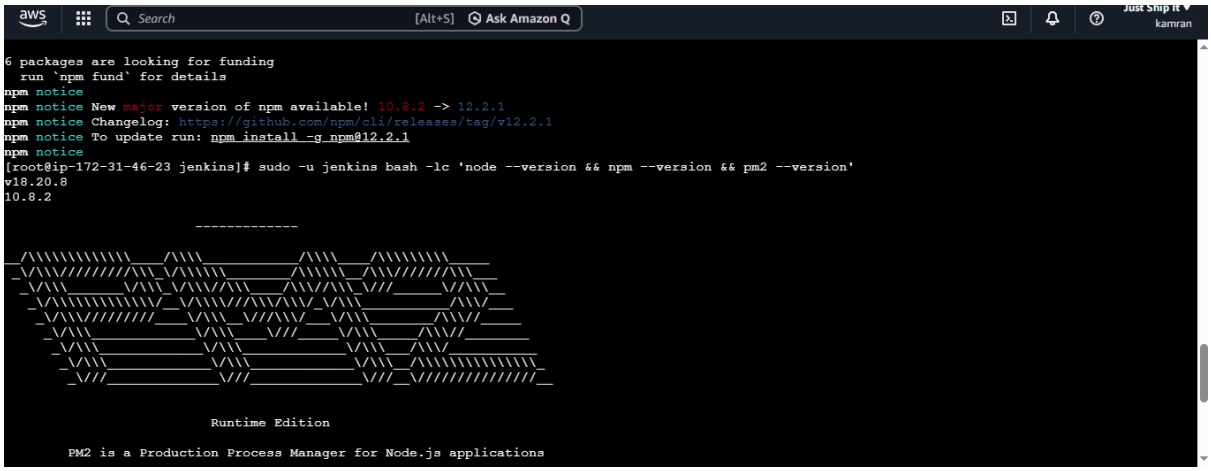
```
output "iam_username" {
  value = aws_iam_user.Admin-user.name
}
resource "aws_iam_user_policy_attachment" "lucy_admin" {
  user       = aws_iam_user.Admin-user.name
  policy_arn = "arn:aws:iam::aws:policy/AdministratorAccess"
}
```

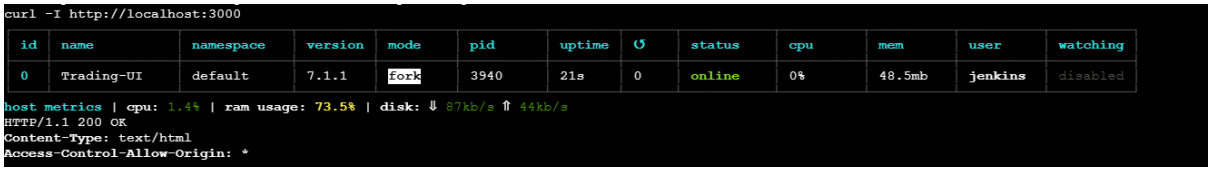


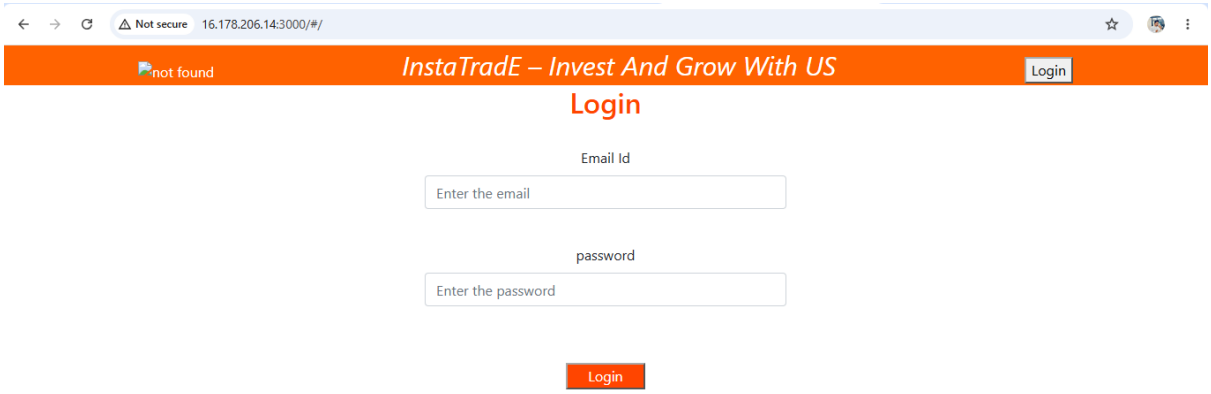


Installed Pm2

- Installed PM2 (Process Manager 2) to manage and serve the Trading-UI application.
- Used PM2 to serve the React production build from the `build` directory.
- Configured the application to run on port `3000`.
- Assigned the process name `Trading-UI` for easy identification.
- Used the `--spa` option to support client-side routing in the React application.
- Troubleshot the PM2 daemon startup issue (`spawn /usr/bin/node-18 EACCES`).
- Verified the deployment using PM2 process status and a local HTTP request.
- PM2 helps manage application processes and supports restarting applications when required.







Jenkins ci/cd

Installed Jenkins

Created pipeline job

Executed pipeline script



```
└──────┴─────────┘
19:32:55  + pm2 save
19:32:56  [32m[PM2] [39mSaving current process list...
19:32:56  [32m[PM2] [39mSuccessfully saved in /var/lib/jenkins/.pm2/dump.pm2
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
19:32:56  Pipeline execution finished.
[Pipeline] echo
19:32:56  Trading-UI pipeline completed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // timestamps
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Maven commands

`1.mvn -version`

Displays the installed Maven version, Java version, and Java home path.

2

`mvn validate`

Checks whether the project structure and configuration are valid.

3

`mvn compile`

Compiles the project's source code.

4

`mvn test`

Runs the project's unit tests.

5

`mvn package`

Packages the compiled code into a JAR or WAR file.

6

```
mvn clean
```

Deletes the generated `target` directory and previous build files.

7

```
mvn install
```

Builds the project and installs the packaged artifact into the local Maven repository.

8

```
mvn deploy
```

Publishes the packaged artifact to a configured remote repository, such as Nexus.

9

```
mvn dependency:tree
```

Displays the project's dependencies in a tree structure.

10

```
mvn site
```

Generates project documentation and reports in the `target/site` directory.